

**Task ID/Title:** generate code for the resume upload feature

**Purpose of AI Use:** Since I had never used Springboot in Java before, I needed AI to help me figure out the right notation to use in addition to my code. I knew about runtimeerror but did not think I didn't think it would work with a website. I didn't need AI for exceptions or building the class since I have already learned how to do it in my other computer engineering classes. The code from the first prompt wasn't really all that useful.

I did not use all the code by AI, I tried to test the code by making a test file. Most of the file made by AI needed to be changed to work with the assignment I didn't need AI for the test file. I changed the functionality of the ApplicationService.java class.

**Chat Link or Prompt/Response:**

Prompt 1:

```
package com.example.sprint;
```

```
import org.w3c.dom.Document;
```

```
public class application {
```

```
    @NotEmpty(message = "Application name is required")
```

```
    private String name;
```

```
    @NotNull(message = "Document is required")
```

```
    private Document document;
```

```
    private boolean isSubmitted;
```

```
    private String status;
```

```
    public application() {
```

```
        this.name = "";
```

```
        this.document = null;
```

```
    this.isSubmitted = false;
    this.status = "Pending";
}
```

```
public application(String name, Document document) {
    this.name = name;
    this.document = document;
    this.isSubmitted = false;
    this.status = "Pending";
}
```

```
public String getName() {
    return name;
}
```

```
public Document getDocument() {
    return document;
}
```

```
public boolean isSubmitted() {
    return isSubmitted;
}
```

```
public String getStatus() {
    return status;
}
```

```
public void submit() {  
    if (isSubmitted) {  
        return;  
    }  
    isSubmitted = true;  
    status = "Submitted";  
}
```

```
public void approve() {  
    if (!isSubmitted || "Approved".equals(status) || "Rejected".equals(status)) {  
        return;  
    }  
    status = "Approved";  
}
```

```
public void reject() {  
    if (!isSubmitted || "Approved".equals(status) || "Rejected".equals(status)) {  
        return;  
    }  
    status = "Rejected";  
}  
}
```

i want to make the other file have the exception such as no empty names, empty doc, if it's already submitted the user can't submit again ect

Response :

Yes. A good way to structure this is to create **custom exception classes** and let your application class throw them when an invalid action happens.

Also, your current @NotEmpty and @NotNull annotations aren't actually enforcing anything by themselves. They only work when a validation framework processes them.

### 1. Create an exception file

You could create one file called ApplicationException.java:

```
package com.example.sprint;
```

```
public class ApplicationException extends RuntimeException {
```

```
    public ApplicationException(String message) {
```

```
        super(message);
```

```
    }
```

```
}
```

You can then use this for all the application-specific errors.

### 2. Modify your application class

I'd change your class to this:

```
package com.example.sprint;
```

```
import org.w3c.dom.Document;
```

```
public class application {
```

```
    private String name;
```

```
    private Document document;
```

```
    private boolean isSubmitted;
```

```
    private String status;
```

```
public application() {  
    this.name = "";  
    this.document = null;  
    this.isSubmitted = false;  
    this.status = "Pending";  
}
```

```
public application(String name, Document document) {  
    if (name == null || name.trim().isEmpty()) {  
        throw new ApplicationException("Application name cannot be empty");  
    }
```

```
    if (document == null) {  
        throw new ApplicationException("Document cannot be null");  
    }
```

```
    this.name = name;  
    this.document = document;  
    this.isSubmitted = false;  
    this.status = "Pending";  
}
```

```
public String getName() {  
    return name;  
}
```

```
public Document getDocument() {  
    return document;  
}
```

```
public boolean isSubmitted() {  
    return isSubmitted;  
}
```

```
public String getStatus() {  
    return status;  
}
```

```
public void submit() {  
    if (isSubmitted) {  
        throw new ApplicationException("Application has already been submitted");  
    }  
  
    isSubmitted = true;  
    status = "Submitted";  
}
```

```
public void approve() {  
    if (!isSubmitted) {  
        throw new ApplicationException("Application must be submitted before it can be  
approved");  
    }  
}
```

```
}
```

```
if ("Approved".equals(status)) {
```

```
    throw new ApplicationException("Application has already been approved");
```

```
}
```

```
if ("Rejected".equals(status)) {
```

```
    throw new ApplicationException("A rejected application cannot be approved");
```

```
}
```

```
status = "Approved";
```

```
}
```

```
public void reject() {
```

```
    if (!isSubmitted) {
```

```
        throw new ApplicationException("Application must be submitted before it can be  
rejected");
```

```
    }
```

```
if ("Rejected".equals(status)) {
```

```
    throw new ApplicationException("Application has already been rejected");
```

```
}
```

```
if ("Approved".equals(status)) {
```

```
    throw new ApplicationException("An approved application cannot be rejected");
```

```
}
```

```

        status = "Rejected";
    }
}

```

### What this gives you

Now these situations produce exceptions:

| Situation                               | Exception            |
|-----------------------------------------|----------------------|
| Name is null                            | ApplicationException |
| Name is ""                              | ApplicationException |
| Name is only spaces                     | ApplicationException |
| Document is null                        | ApplicationException |
| Submit twice                            | ApplicationException |
| Approve before submitting               | ApplicationException |
| Reject before submitting                | ApplicationException |
| Approve an already approved application | ApplicationException |
| Reject an already rejected application  | ApplicationException |
| Approve a rejected application          | ApplicationException |
| Reject an approved application          | ApplicationException |

### One important distinction

You **don't need ResponseEntity** for this part.

ResponseEntity is useful later if you have a Spring REST controller, for example:

```

@PostMapping("/applications/{id}/submit")
public ResponseEntity<?> submitApplication(...) {
    ...
}

```

```
}
```

Your application class itself should just throw the exception. Then your **controller** can decide how that exception becomes an HTTP response.

Also, since you're using:

```
import org.w3c.dom.Document;
```

you should **not** use `.body()` or `.html()` on `Document`. Those methods belong to Jsoup's `Document`, not `org.w3c.dom.Document`.

I had already tried to make a `ApplicationController.java` before asking AI to help me. I only needed to properly debug my file. I also did not follow the code word for word. I have changed a few things but I needed a base to help me understand Springboot.

Prompt 2: what if i keep the application file the same and build off of this "package com.example.sprint;

```
import org.springframework.http.ResponseEntity;
```

```
import org.springframework.web.bind.annotation.PathVariable;
```

```
import org.springframework.web.bind.annotation.PostMapping;
```

```
import org.springframework.web.bind.annotation.RequestMapping;
```

```
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
@RequestMapping("/api/applications")
```

```
public class ApplicationController {
```

```
    private final ApplicationService applicationService;
```

```
    public ApplicationController(ApplicationService applicationService) {
```

```
        this.applicationService = applicationService;
```

```

    }

    @PostMapping("/{id}/approve")
    public ResponseEntity<String> approveApplication(@PathVariable Long id) {
        applicationService.approveApplication(id);
        return ResponseEntity.ok("Application approved successfully.");
    }
}
"

```

Response : Yes — **that's actually a cleaner approach** if your goal is to keep application.java as a simple model/entity and put the validation/business rules in your service/controller layer.

Your architecture can look like:

application.java

↓

ApplicationService.java

↓

ApplicationController.java

↓

ApplicationException.java

The application class stays mostly unchanged, while ApplicationService checks things like "already submitted."

## 1. Keep your application.java

You can keep your current class as-is.

One thing to be aware of: your annotations:

@NotEmpty

@NotNull

will require the appropriate validation imports/dependency if they're currently giving errors:

```
import jakarta.validation.constraints.NotEmpty;
```

```
import jakarta.validation.constraints.NotNull;
```

But those annotations don't automatically handle your business rules like "can't submit twice."

---

## 2. Create ApplicationException.java

Create a separate file:

```
package com.example.sprint;
```

```
public class ApplicationException extends RuntimeException {
```

```
    public ApplicationException(String message) {
```

```
        super(message);
```

```
    }
```

```
}
```

This gives you your custom exception type.

---

## 3. Create ApplicationService.java

This is where I'd put your rules.

For example:

```
package com.example.sprint;
```

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class ApplicationService {

    public void approveApplication(Long id) {

        // For now, this is just an example.
        // Later, you will retrieve the application from your database.

        application app = findApplication(id);

        if (app == null) {
            throw new ApplicationException("Application not found.");
        }

        if (!app.isSubmitted()) {
            throw new ApplicationException(
                "Application must be submitted before it can be approved."
            );
        }

        if ("Approved".equals(app.getStatus())) {
            throw new ApplicationException(
                "Application has already been approved."
            );
        }

        if ("Rejected".equals(app.getStatus())) {
```

```
        throw new ApplicationException(
            "A rejected application cannot be approved."
        );
    }
```

```
    app.approve();
}
```

```
private Application findApplication(Long id) {
    // Temporary placeholder
    return null;
}
}
```

The findApplication() part would eventually be replaced with something like a repository/database lookup.

---

#### **4. Your controller can stay like this**

Your existing controller is already structured correctly:

```
package com.example.spring;
```

```
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
```

```

@RestController
@RequestMapping("/api/applications")
public class ApplicationController {

    private final ApplicationService applicationService;

    public ApplicationController(ApplicationService applicationService) {
        this.applicationService = applicationService;
    }

    @PostMapping("/{id}/approve")
    public ResponseEntity<String> approveApplication(@PathVariable Long id) {
        applicationService.approveApplication(id);
        return ResponseEntity.ok("Application approved successfully.");
    }
}

```

### **But there's an important problem**

Right now, if ApplicationService throws:

```
throw new ApplicationException("Application not found.");
```

Spring will return an error response, but you don't yet have control over **what HTTP status and message** the client receives.

You can create an exception handler for that.

For example, create:

ApplicationExceptionHandler.java

```
package com.example.spring;
```

```
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
```

```
@RestControllerAdvice
```

```
public class ApplicationExceptionHandler {
```

```
    @ExceptionHandler(ApplicationException.class)
```

```
    public ResponseEntity<String> handleApplicationException(
        ApplicationException exception) {
```

```
        return ResponseEntity
```

```
            .badRequest()
```

```
            .body(exception.getMessage());
```

```
    }
```

```
}
```

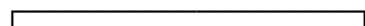
Now your structure is:

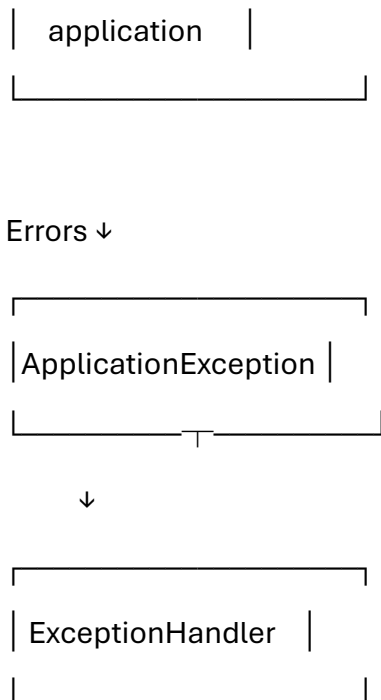


↓



↓





This is generally a better separation of responsibilities:

- **application** → stores application data/state.
- **ApplicationService** → performs application operations and business rules.
- **ApplicationException** → represents application-specific errors.
- **ApplicationExceptionHandler** → converts those errors into HTTP responses.
- **ApplicationController** → handles HTTP requests.

So yes, **you can absolutely keep your application.java unchanged and build around it.**

**Validation:** How the output was verified (e.g., manual review, testing, peer review, static analysis, CI/CD checks, comparison with requirements). For this prompt I made a demo file to test the outputs by using the functions in ApplicationServie.

**Decision:** The output was modified

**Reflection:** I learned about the uses of Springboot in java and how exception could be programmed in Java for a webapp

**Responsible Person:** Shanza Riasat 40337845

**Secon use of AI**

**Task ID/Title:** backend of resume upload

**Purpose of AI Use:** Debug and generate code to suit the backend of resume upload

**Chat Link or Prompt/Response:**

Prompt 1: "is the code enough to work as a resume upload back end "import  
org.springframework.stereotype.Service;

import java.util.Optional;

@Service

public class ApplicationService{

public void approveApplication(Long id) {

Optional<Application> applicationOptional = ApplicationRepository.findById(id);

if (!applicationOptional.isPresent()) {

throw new ApplicationException("Application not found.");

}

Application app = applicationOptional.get();

if(app.getName() == null || app.getName().isEmpty()) {

throw new ApplicationException("Application name is required.");

}

if(app.getDocument() == null) {

throw new ApplicationException("Document is required.");

}

if (!app.isSubmitted() || "Pending".equals(app.getStatus())) {

throw new ApplicationException("Application must be submitted before approval.");

}

if (app.isSubmitted() && "Approved".equals(app.getStatus())) {

```

        throw new ApplicationException("Application is already approved.");
    }

    if (app.isSubmitted() && "Rejected".equals(app.getStatus())) {

        throw new ApplicationException("Application has been rejected and cannot be
approved.");
    }

    app.approve();
}

public void submitApplication(Long id) {

    Optional<Application> applicationOptional = ApplicationRepository.findById(id);
    if (!applicationOptional.isPresent()) {

        throw new ApplicationException("Application not found.");
    }

    Application app = applicationOptional.get();
    if (app.getDocument() == null) {

        throw new ApplicationException("Document is required.");
    }

    if (app.isSubmitted()) {

        throw new ApplicationException("Application has already been submitted.");
    }

    if ("Approved".equals(app.getStatus()) || "Rejected".equals(app.getStatus())) {

        throw new ApplicationException("Application has already been processed and
cannot be submitted.");
    }

    app.submit();
}

```

```

    }

    public void rejectApplication(Long id) {
        Optional<Application> applicationOptional = ApplicationRepository.findById(id);
        if (!applicationOptional.isPresent()) {
            throw new ApplicationException("Application not found.");
        }
        Application app = applicationOptional.get();
        if (!app.isSubmitted() || "Pending".equals(app.getStatus())) {
            throw new ApplicationException("Application must be submitted before rejection.");
        }
        if (app.isSubmitted() && "Approved".equals(app.getStatus())) {
            throw new ApplicationException("Application has already been approved and cannot
be rejected.");
        }
        if (app.isSubmitted() && "Rejected".equals(app.getStatus())) {
            throw new ApplicationException("Application has already been rejected.");
        }
        app.reject();
    }
}

```

""

```

import org.w3c.dom.Document;

import jakarta.validation.constraints.NotEmpty;

```

```
import jakarta.validation.constraints.NotNull;

public class Application {

    @NotEmpty(message = "Application name is required")
    private String name;

    @NotNull(message = "Document is required")
    private long id;

    private Document document;

    private boolean isSubmitted;

    private String status;

    public Application() {
        this.name = "";
        this.document = null;
        this.isSubmitted = false;
        this.status = "Pending";
    }

    public Application(String name, long id, Document document) {
        this.name = name;
        this.id = id;
        this.document = document;
        this.isSubmitted = false;
        this.status = "Pending";
    }
}
```

```
public String getName() {  
    return name;  
}
```

```
public Document getDocument() {  
    return document;  
}
```

```
public long getId() {  
    return id;  
}
```

```
public boolean isSubmitted() {  
    return isSubmitted;  
}
```

```
public String getStatus() {  
    return status;  
}
```

```
public void submit() {  
    if (isSubmitted) {  
        return;  
    }  
    isSubmitted = true;  
    status = "Submitted";  
}
```

```
public void approve() {  
    if (!isSubmitted || "Approved".equals(status) || "Rejected".equals(status)) {  
        return;  
    }  
    status = "Approved";  
}
```

```
public void reject() {  
    if (!isSubmitted || "Approved".equals(status) || "Rejected".equals(status)) {  
        return;  
    }  
    status = "Rejected";  
}  
}  
"
```

```
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.PathVariable;  
import org.springframework.web.bind.annotation.PostMapping;  
import org.springframework.web.bind.annotation.RequestMapping;  
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
@RequestMapping("/api/applications")
```

```
public class ApplicationController {
```

```
private final ApplicationService applicationService;
```

```
public ApplicationController(ApplicationService applicationService) {
```

```
    this.applicationService = applicationService;
```

```
}
```

```
@PostMapping("/{id}/submit")
```

```
public ResponseEntity<String> submitApplication(@PathVariable Long id) {
```

```
    applicationService.submitApplication(id);
```

```
    return ResponseEntity.ok("Application submitted successfully.");
```

```
}
```

```
@PostMapping("/{id}/approve")
```

```
public ResponseEntity<String> approveApplication(@PathVariable Long id) {
```

```
    applicationService.approveApplication(id);
```

```
    return ResponseEntity.ok("Application approved successfully.");
```

```
}
```

```
@PostMapping("/{id}/reject")
```

```
public ResponseEntity<String> rejectApplication(@PathVariable Long id) {
```

```
    applicationService.rejectApplication(id);
```

```
    return ResponseEntity.ok("Application rejected successfully.");
```

```
}
```

```
}
```

```
""
```

Response :“ Not quite. **Your code has the basic application workflow**, but it is not yet enough to function as a real resume-upload backend.

You currently have something closer to an **application status-management backend**:

Pending → Submitted → Approved/Rejected

The biggest issue is that your document field is not actually set up to receive and store an uploaded resume.

### **What you already have**

Your backend does have useful pieces:

- Application model
- ApplicationService
- ApplicationController
- submit()
- approve()
- reject()
- ID-based endpoints:
  - POST /api/applications/{id}/submit
  - POST /api/applications/{id}/approve
  - POST /api/applications/{id}/reject
- Validation/error checks
- Application status tracking

So the **workflow logic is there**.

### **What is missing for a resume upload**

#### **1. Document is the wrong type**

You currently have:

```
import org.w3c.dom.Document;
```

```
private Document document;
```

org.w3c.dom.Document represents an **XML document**, not an uploaded PDF/DOCX resume.

For a simple resume-upload backend, you'd normally receive something like:

MultipartFile resume

in the controller.

For example:

```
@PostMapping("/{id}/resume")
public ResponseEntity<String> uploadResume(
    @PathVariable Long id,
    @RequestParam("resume") MultipartFile resume) {

    applicationService.uploadResume(id, resume);

    return ResponseEntity.ok("Resume uploaded successfully.");
}
```

---

## 2. You don't currently have an upload endpoint

Your controller only has:

POST /{id}/submit

POST /{id}/approve

POST /{id}/reject

You need something like:

POST /{id}/resume

to actually upload the file.

---

## 3. Your id validation is incorrect

You have:

```
@NotNull
```

```
private long id;
```

long cannot be null, so @NotNull doesn't make sense here.

If you want @NotNull, use:

```
private Long id;
```

Also, if this is a database entity, you'd normally have something like:

```
@Id
```

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Long id;
```

---

#### **4. You aren't saving changes**

This is important.

You do:

```
app.submit();
```

or:

```
app.approve();
```

but I don't see:

```
ApplicationRepository.save(app);
```

So if ApplicationRepository isn't using JPA's managed entity behavior in a transaction, your changes won't necessarily be persisted.

A typical service would have:

```
private final ApplicationRepository applicationRepository;
```

```
public ApplicationService(ApplicationRepository applicationRepository) {
```

```
    this.applicationRepository = applicationRepository;
```

```
}
```

and then:

```
applicationRepository.save(app);
```

---

## 5. Your approval condition has a logical problem

You have:

```
if (!app.isSubmitted() || "Pending".equals(app.getStatus())) {  
    throw new ApplicationException(  
        "Application must be submitted before approval."  
    );  
}
```

But your submit() method changes:

```
status = "Submitted";
```

So that's okay for a newly submitted application.

However, conceptually, you don't really need to check both isSubmitted **and** the status string. You're storing the same state in two different ways.

You could simplify your model to:

Pending

Submitted

Approved

Rejected

and use status as the source of truth.

That would eliminate:

```
private boolean isSubmitted;
```

entirely.

---

## A more realistic architecture

For a resume-upload backend, I'd structure it approximately like this:

Application

|

├— id

├— name

├— resume

├— status

└— ...

↓

ApplicationController

↓

ApplicationService

↓

ApplicationRepository

↓

Database

And the API could be:

POST /api/applications

POST /api/applications/{id}/resume

POST /api/applications/{id}/submit

POST /api/applications/{id}/approve

POST /api/applications/{id}/reject

GET /api/applications/{id}

For example:

**Create application**

POST /api/applications

### **Upload resume**

POST /api/applications/123/resume

with:

Content-Type: multipart/form-data

resume = resume.pdf

### **Submit**

POST /api/applications/123/submit

### **Approve**

POST /api/applications/123/approve

### **One other important point**

If your assignment specifically asks you to demonstrate **Spring Boot backend functionality**, you probably don't need to build a sophisticated file-storage system yet.

You could initially store the resume as:

```
private byte[] document;
```

in the database, or store a file path/URL if you're using external file storage.

For a real production application, you'd usually store the actual file in object/file storage and keep its metadata/path in the database.

**So the answer is: your code is enough for the application workflow, but not enough for an actual resume-upload backend.** The main things you need are a real upload mechanism (MultipartFile), an upload endpoint, persistence, and a proper database entity/repository setup.

”

Prompt 2: “how would you change the following files to make it work”

Response 2 : “Yes. I'd keep your current **submit / approve / reject workflow**, but change the model so it can actually receive a resume file and persist the application.

Below is a simple version that stays close to your current code.

### **1. Application.java**

Replace your current class with:

```
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.validation.constraints.NotEmpty;
```

```
@Entity
```

```
public class Application {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    @NotEmpty(message = "Application name is required")
```

```
    private String name;
```

```
    private byte[] document;
```

```
    private boolean submitted;
```

```
    private String status;
```

```
    public Application() {
```

```
        this.status = "Pending";
```

```
        this.submitted = false;
```

```
}
```

```
public Application(String name) {
```

```
    this.name = name;
```

```
    this.status = "Pending";
```

```
    this.submitted = false;
```

```
}
```

```
public Long getId() {
```

```
    return id;
```

```
}
```

```
public String getName() {
```

```
    return name;
```

```
}
```

```
public byte[] getDocument() {
```

```
    return document;
```

```
}
```

```
public boolean isSubmitted() {
```

```
    return submitted;
```

```
}
```

```
public String getStatus() {
```

```
    return status;
```

```
}
```

```
public void uploadDocument(byte[] document) {  
    this.document = document;  
}
```

```
public void submit() {  
    if (!submitted) {  
        submitted = true;  
        status = "Submitted";  
    }  
}
```

```
public void approve() {  
    if (submitted && !"Approved".equals(status)  
        && !"Rejected".equals(status)) {  
        status = "Approved";  
    }  
}
```

```
public void reject() {  
    if (submitted && !"Approved".equals(status)  
        && !"Rejected".equals(status)) {  
        status = "Rejected";  
    }  
}
```

```
}
```

## 2. ApplicationRepository.java

You need a repository if you don't already have one:

```
import org.springframework.data.jpa.repository.JpaRepository;
```

```
public interface ApplicationRepository extends JpaRepository<Application, Long> {  
}
```

This gives you:

```
findById(id)
```

```
save(application)
```

```
delete(application)
```

```
findAll()
```

without having to implement them yourself.

---

## 3. ApplicationService.java

I'd change your service to this:

```
import org.springframework.stereotype.Service;
```

```
import org.springframework.web.multipart.MultipartFile;
```

```
import java.io.IOException;
```

```
import java.util.Optional;
```

```
@Service
```

```
public class ApplicationService {
```

```
    private final ApplicationRepository applicationRepository;
```

```
public ApplicationService(ApplicationRepository applicationRepository) {  
    this.applicationRepository = applicationRepository;  
}
```

```
public void uploadResume(Long id, MultipartFile file) {
```

```
    Optional<Application> applicationOptional =  
        applicationRepository.findById(id);
```

```
    if (!applicationOptional.isPresent()) {  
        throw new ApplicationException("Application not found.");  
    }
```

```
    Application app = applicationOptional.get();
```

```
    if (app.isSubmitted()) {  
        throw new ApplicationException(  
            "Cannot change the resume after submission."  
        );  
    }
```

```
    if (file == null || file.isEmpty()) {  
        throw new ApplicationException("Resume file is required.");  
    }
```

```
try{
    app.uploadDocument(file.getBytes());
} catch (IOException e) {
    throw new ApplicationException(
        "Could not upload resume."
    );
}

applicationRepository.save(app);
}

public void submitApplication(Long id) {

    Application app = findApplication(id);

    if (app.getName() == null || app.getName().isEmpty()) {
        throw new ApplicationException(
            "Application name is required."
        );
    }

    if (app.getDocument() == null) {
        throw new ApplicationException(
            "Resume is required before submission."
        );
    }
}
```

```
if (app.isSubmitted()) {  
    throw new ApplicationException(  
        "Application has already been submitted."  
    );  
}
```

```
if ("Approved".equals(app.getStatus())  
    || "Rejected".equals(app.getStatus())) {  
    throw new ApplicationException(  
        "Application has already been processed."  
    );  
}
```

```
app.submit();
```

```
applicationRepository.save(app);  
}
```

```
public void approveApplication(Long id) {
```

```
    Application app = findApplication(id);
```

```
    if (!app.isSubmitted()) {  
        throw new ApplicationException(  
            "Application must be submitted before approval."  
        );  
    }
```

```
);  
}
```

```
if ("Approved".equals(app.getStatus())) {  
    throw new ApplicationException(  
        "Application is already approved."  
    );  
}
```

```
if ("Rejected".equals(app.getStatus())) {  
    throw new ApplicationException(  
        "Application has been rejected and cannot be approved."  
    );  
}
```

```
app.approve();
```

```
applicationRepository.save(app);  
}
```

```
public void rejectApplication(Long id) {
```

```
    Application app = findApplication(id);
```

```
    if (!app.isSubmitted()) {  
        throw new ApplicationException(  
            "Application is not submitted."  
        );  
    }  
}
```

```
        "Application must be submitted before rejection."
    );
}

if ("Approved".equals(app.getStatus())) {
    throw new ApplicationException(
        "Application has already been approved and cannot be rejected."
    );
}

if ("Rejected".equals(app.getStatus())) {
    throw new ApplicationException(
        "Application has already been rejected."
    );
}

app.reject();

applicationRepository.save(app);
}
```

```
private Application findApplication(Long id) {

    Optional<Application> applicationOptional =
        applicationRepository.findById(id);
```

```

    if (!applicationOptional.isPresent()) {
        throw new ApplicationException("Application not found.");
    }

    return applicationOptional.get();
}
}

```

Notice that I moved the repeated:

`ApplicationRepository.findById(id)`

into:

`findApplication(id)`

This makes the service much cleaner.

---

#### 4. ApplicationController.java

Now add an endpoint for uploading the resume:

```

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.multipart.MultipartFile;

```

```
@RestController
```

```
@RequestMapping("/api/applications")
```

```
public class ApplicationController {
```

```
private final ApplicationService applicationService;
```

```
public ApplicationController(ApplicationService applicationService) {  
    this.applicationService = applicationService;  
}
```

```
@PostMapping("/{id}/resume")
```

```
public ResponseEntity<String> uploadResume(  
    @PathVariable Long id,  
    @RequestParam("resume") MultipartFile resume) {
```

```
    applicationService.uploadResume(id, resume);
```

```
    return ResponseEntity.ok(  
        "Resume uploaded successfully."  
    );
```

```
}
```

```
@PostMapping("/{id}/submit")
```

```
public ResponseEntity<String> submitApplication(  
    @PathVariable Long id) {
```

```
    applicationService.submitApplication(id);
```

```
    return ResponseEntity.ok(
```

```
        "Application submitted successfully."
    );
}
```

```
@PostMapping("/{id}/approve")
public ResponseEntity<String> approveApplication(
    @PathVariable Long id) {

    applicationService.approveApplication(id);

    return ResponseEntity.ok(
        "Application approved successfully."
    );
}
```

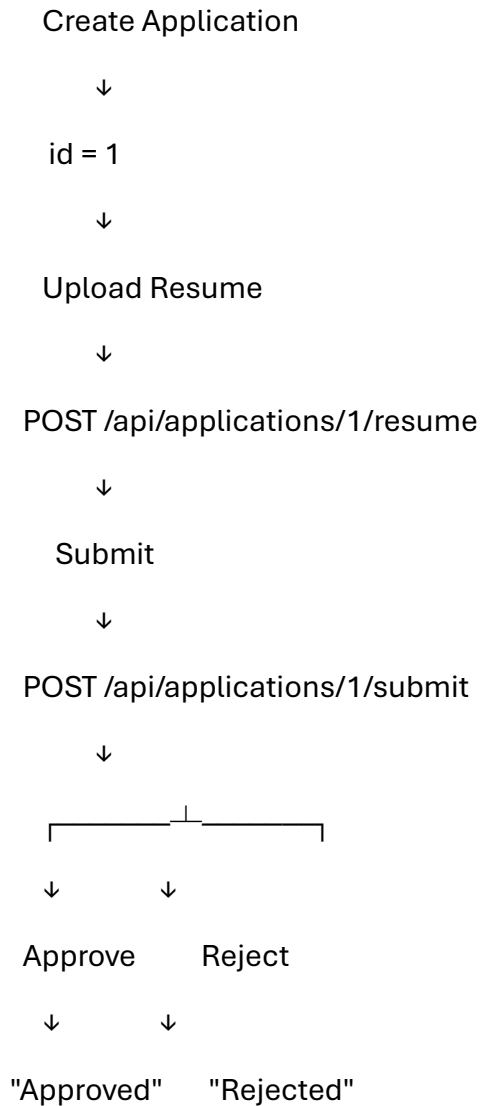
```
@PostMapping("/{id}/reject")
public ResponseEntity<String> rejectApplication(
    @PathVariable Long id) {

    applicationService.rejectApplication(id);

    return ResponseEntity.ok(
        "Application rejected successfully."
    );
}
}
```

## 5. The workflow is now

Your backend can work like this:



For example, you could send:

POST /api/applications/1/resume

with a multipart/form-data body:

resume = my\_resume.pdf

Then:

POST /api/applications/1/submit

Then an administrator could call:

POST /api/applications/1/approve

or:

POST /api/applications/1/reject

### **One thing you still need**

You need the JPA/database dependencies in your pom.xml, something along the lines of:

```
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-data-jpa</artifactId>
```

```
</dependency>
```

```
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-web</artifactId>
```

```
</dependency>
```

```
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-validation</artifactId>
```

```
</dependency>
```

and a database driver such as H2 for testing or MySQL/PostgreSQL for a real database.

**One important change from your original design:** don't use `org.w3c.dom.Document` for the resume. `MultipartFile` receives the uploaded PDF/DOCX, and `byte[]` document stores its contents in this simple implementation.

”

**AI-Suggested Content:** It proposed to implement an JPA repository for the data base and change the exception handling

**Validation:** I made a testing file to test the status of the application process. For example, testing if an application with not be rejected after it was already approved

**Decision:** I modified some of the code

**Reflection:** I learned how to use the some Springboot utilities and the JRA repository

**Responsible Person:** Shanza Riasat 40337845